SEMITEC FOUNDATION · FPGA Starter Track

Lab 1: A 4-bit Shift Register on the FPGA

Lab at a glance

Duration	Difficulty	Work in	Version
about 2 hours	**Beginner**	**individually**	**1.1**

Equipment	AMD AUP-ZU3 (or any FPGA board with LEDs, a push button and a switch), USB cable
Software	AMD Vivado 2024.2 or newer
Prerequisites	Training *Shift registers* (SemiTec Academy); basic SystemVerilog
Author	Arman Manukyan, PhD
Department	SemiTec Foundation

Contents

1 Overview 1

2 Background 2

3 Preparation 2

4 Procedure 3

1 Create the Vivado project . . . 3
2 Write the shift register . . . 3
3 Simulate with a testbench . . . 3
4 Debounce the button . . . 4
5 Constraints, implementation and timing . . . 4
6 Run it on the board . . . 4

5 Results 4

6 Questions 4

7 Report and assessment 5

8 References and further reading 5

1. Overview

You describe a 4-bit *serial-in, parallel-out* (SIPO) shift register in SystemVerilog, verify it with a self-checking testbench and run it on the board: a push button gives the shift pulse, a switch the serial input, and four LEDs show the register. The same structure receives bits in UART and SPI interfaces and in JTAG data registers.

Objectives

After this lab you can:

- describe sequential logic with `always_ff`;
- verify a module with a self-checking testbench and an assertion;
- explain why a push button must be synchronised and debounced, and do it;
- map ports to pins, implement the design and check timing.

2. Background

A shift register is a chain of D flip-flops sharing one clock (Figure 1). On every active edge each flip-flop takes the value of its left neighbour and the first one takes the serial input [1]. After four edges the four bits that entered one after another are available in parallel.

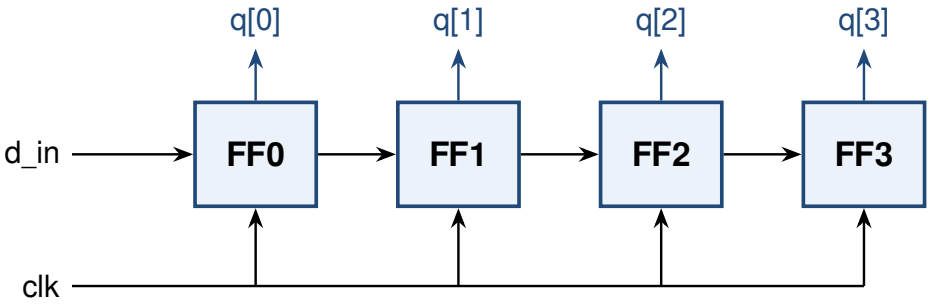


Figure 1: 4-bit serial-in, parallel-out shift register

A mechanical push button *bounces*: for a few milliseconds its contact opens and closes many times. Used directly as a clock it gives several edges per press. It is also asynchronous to the board clock and must be synchronised with two flip-flops before use.

3. Preparation

- Complete the training *Shift registers* in SemiTec Academy.
- Install Vivado and the board files of your board.
- Download the lab files: open this lab in SemiTec Academy → **Files** → *Download all*. You get `src/rtl`, `src/sim` and `src/constraints`.

Safety

Touch the board only at its edges and discharge yourself (ESD) before handling it. Connect and disconnect cables only when the board is powered off.

4. Procedure

Step 1 Create the Vivado project

Create an RTL project for your board. Add `src/rtl/*.sv` as design sources and `src/sim/tb_shift4.sv` as a simulation source; set `top` as the top module.

Step 2 Write the shift register

Files: `src/rtl/shift4.sv`

Complete the module. On reset all bits become 0; otherwise the bits move one place and `d_in` enters bit 0:

```
always_ff @(posedge clk)
  if (rst) q <= '0;
  else     q <= {q[2:0], d_in};
```

Step 3 Simulate with a testbench

Files: `src/sim/tb_shift4.sv`

Run the behavioural simulation of `tb_shift4`. It shifts in 1, 0, 1, 1 and checks the result:

src/sim/tb_shift4.sv self-checking testbench

```
// tb_shift4.sv — self-checking testbench for shift4 (Lab 1, step 3)
`timescale 1ns/1ps
module tb_shift4;
  logic clk = 0, rst = 1, d_in = 0;
  logic [3:0] q;

  shift4 dut (.*);
  always #5 clk = ~clk;            // 100 MHz

  task automatic shift_in(input logic b);
    d_in = b;
    @(posedge clk); #1;
  endtask

  initial begin
    repeat (2) @(posedge clk);
    rst = 0;
    shift_in(1); shift_in(0); shift_in(1); shift_in(1);
    assert (q == 4'b1011) else $error("q = %b, expected 1011", q);
    $display("TEST %s", (q == 4'b1011) ? "PASSED" : "FAILED");
    $finish;
  end
endmodule
```

Expected result: the log prints `TEST PASSED`; in the waveform `q` becomes `1011` after the fourth edge.

Step 4 Debounce the button

Files: `src/rtl/debounce.sv` `src/rtl/top.sv`

Study `debounce.sv`: a two flip-flop synchroniser and a counter that accepts a new level only after it was stable for 10 ms (10^6 clocks at 100 MHz). In `top.sv` replace the button clock by a clock enable: clock the shift register with `sysclk` and shift only when `shift_en` is 1.

Step 5 Constraints, implementation and timing

Files: `src/constraints/top.xdc`

Complete the pin names in `top.xdc` from the master XDC of your board (button, switch, four LEDs, clock). Run synthesis and implementation and open the timing summary.

Expected result: no failing timing paths (WNS ≥ 0).

Step 6 Run it on the board

Generate the bitstream and program the device. Set SW0, press the button, watch the bit move along the LEDs.

✓ Checkpoint

After entering 1, 0, 1, 1 the LEDs show `1011` (LED3 is the first bit) — show it to the teacher.

5. Results

Table 1: Results to record

Quantity	Expected	Measured	Unit
LEDs after entering 1, 0, 1, 1	1011	_____	—
Shifts per button press, without debouncing	1	_____	shifts
Shifts per button press, with debouncing	1	_____	shifts
Worst negative slack (WNS)	≥ 0	_____	ns

6. Questions

Q1. Which bit receives `d_in` in `q <= {q[2:0], d_in};`? Which bit is lost?

Q2. Why is the button synchronised with two flip-flops before the debouncer?

Q3. What happens to the register if the button is used directly as a clock? Explain with your measurement.

Q4. How would you change the design to a *parallel-in, serial-out* register?

7. Report and assessment

Hand in: `shift4.sv`, `top.sv`, `top.xdc`; a screenshot of the simulation with `TEST PASSED`; the timing summary; a photo of the LEDs showing `1011`; the answers to the questions. In SemiTec Academy tick all steps and pass the check quiz.

Criterion	Weight
Working design on the board (checkpoint)	40 %
Testbench passes, waveform explained	25 %
Debouncing and clock enable done correctly	15 %
Answers to the questions	20 %

8. References and further reading

[1] Sarah L. Harris and David Money Harris. *Digital Design and Computer Architecture*. RISC-V. Morgan Kaufmann, 2022.

[2] *Vivado Design Suite User Guide: Synthesis*. UG901. AMD.